30-Day Android Challenge

Senior Android interview prep — Junior → Mid → Senior → Architect

2026 Edition

DAY 20

Security — Part 2: R8, Integrity, Biometrics, Passkeys & MASVS



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20

Why Part 2 matters in 2026

Part 1 closed the doors. Part 2 is about the device you **don't control** — and how to stay safe anyway by moving trust into **hardware keys** and **server-side verification**:

- R8 raises the cost of reverse engineering — it doesn't hide secrets
- Root and tamper checks are **signals**, not decisions
- SafetyNet Attestation is retired — **Play Integrity** replaced it; key attestation covers devices without Play
- A fingerprint means something only when it unlocks a **Keystore key** (`CryptoObject`)
- **Passkeys** through Credential Manager are replacing passwords
- OWASP MASVS gives teams a shared checklist; the **supply chain** is the new attack surface

Level	Questions
Junior	What R8 really does · Root & tamper detection · Biometric basics
Mid	BiometricPrompt + CryptoObject · Passkeys · Play Integrity · Sensitive screens
Senior	Verifying integrity on the server · Key attestation · Responding to tampering
Architect	Defense in depth · OWASP MASVS & MASTG · Supply-chain security



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20

Mental model: raise the cost, decide on the server



- A determined device owner can **read, patch and hook** everything in your APK
- Client checks make attacks slower and produce signals — they are never the final word
- Real guarantees come from **keys in secure hardware** and **checks the attacker can't skip**

THE ONE QUESTION TO ASK

"Who verifies this?" — if the answer is "the app", assume an attacker can skip it.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20

→

Junior

What R8 really does · Root & tamper detection · Biometric basics



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What R8 really does

```
android {
    buildTypes {
        release {
            isMinifyEnabled = true    // shrink + obfuscate
            isShrinkResources = true
            proguardFiles(
                getDefaultProguardFile(
                    "proguard-android-optimize.txt"),
                "proguard-rules.pro",
            )
        }
    }
}
```

R8 does	R8 does not
Remove unused code (shrinking)	Encrypt strings, URLs or API keys
Rename classes and members (obfuscation)	Hide resources, the manifest or native libraries
Optimize and inline code	Stop a debugger or Frida at runtime
Write `mapping.txt` for readable stack traces	Protect logic you shipped to the client



Esmaeeil Moradi
@esmaeeil-moradi700

Q1. What interviewers check

INTERVIEW TRAP

"R8 hides my API key." — String constants stay **in plain text** in the DEX; a decompiler like jadx shows them in minutes. Secrets belong on the server; client keys must be restricted and treated as public (Day 12).

INTERVIEW TRAP

"We use ProGuard." — R8 replaced ProGuard as the default compiler back in AGP 3.4. It reads the same `-keep` rules, but it's a different tool — and full mode is the default since AGP 8 (Day 18).

FOLLOW-UP

"How do you read a production crash?" — Keep `mapping.txt` for every release (upload it to Play Console and your crash reporter) and retrace. Losing it means unreadable stack traces for that version forever.



Q2. Root and tamper detection — and its limits

Client check	Looks for	Typical bypass
Root artifacts	`su` binaries, root-manager apps	Root-hiding modules hide them from chosen apps
Build properties	`test-keys`, `ro.debuggable`	Spoofed properties
Debugger / hooking	`Debug.isDebuggerConnected()`, Frida traces	The check itself gets hooked
App signature / installer	Re-signed or sideloaded APK	Patched out of the DEX

Every one of these runs **inside code the attacker controls**. Use several, report them to your server — and let server-verified signals make the decision (Q6, Q8).

INTERVIEW TRAP

"Refuse to run on rooted devices, emulators, VPNs or with developer options on." — Last year's version recommended exactly this. VPNs and developer options are everyday tools for real users; blocking them punishes customers, while attackers bypass the check in an afternoon.



Esmaeeil Moradi
@esmaeeil-moradi700

Q2. A signature self-check — and its trap

```kotlin
fun signingCertSha256(context: Context): List<String> {
    val info = context.packageManager.getPackageInfo(
        context.packageName,
        PackageManager.GET_SIGNING_CERTIFICATES, // API 28+
    )
    val signers = info.signingInfo?.apkContentsSigners.orEmpty()
    return signers.map { sig ->
        MessageDigest.getInstance("SHA-256")
            .digest(sig.toByteArray())
            .joinToString("") { "%02x".format(it) }
    }
}
```

INTERVIEW TRAP

"Compare it with our keystore's fingerprint." — With **Play App Signing**, Google re-signs what users install with the **app signing key**, not your upload key. Hard-code the upload key and the check fails for every Play user. Use the app-signing certificate from Play Console.

INTERVIEW TRAP

"Hash the DEX at runtime and compare." — The comparison lives in the same DEX the attacker patches. Play Integrity's app verdict (Q6) does this check where they can't reach it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. Biometric authentication basics

```
val allowed = BIOMETRIC_STRONG or DEVICE_CREDENTIAL
when (BiometricManager.from(context).canAuthenticate(allowed)) {
    BiometricManager.BIOMETRIC_SUCCESS -> showPrompt()
    BiometricManager.BIOMETRIC_ERROR_NONE_ENROLLED ->
        offerEnrollment() // Settings.ACTION_BIOMETRIC_ENROLL
    else -> usePasswordInstead()
}

val info = BiometricPrompt.PromptInfo.Builder()
    .setTitle("Unlock your records")
    .setAllowedAuthenticators(allowed)
    .build() // no negative button when DEVICE_CREDENTIAL is set
```

Authenticator	Meaning	Use for
`BIOMETRIC_STRONG` (Class 3)	Strong spoof resistance; can unlock Keystore keys	Payments, key access
`BIOMETRIC_WEAK` (Class 2)	Weaker; **not** allowed with a `CryptoObject`	Convenience unlock
`DEVICE_CREDENTIAL`	PIN, pattern or password	Fallback



Esmaeeil Moradi
@esmaeeil-moradi700

Q3. What interviewers check

INTERVIEW TRAP

"`onAuthenticationSucceeded` *means the user is verified."* — Without a `CryptoObject` it's just a callback in your process. A hooking tool can call it directly. For anything valuable, bind the prompt to a key (Q4).

INTERVIEW TRAP

"We use `FingerprintManager`*."* — Deprecated since API 28. `BiometricPrompt` (AndroidX) covers face and other modalities, provides the system UI and backports to older versions.

FOLLOW-UP

"Can every API level combine `BIOMETRIC_STRONG` *with* `DEVICE_CREDENTIAL`*?"* — No. That combination isn't supported on API 28–29, and `DEVICE_CREDENTIAL` alone needs API 30+. Check with `canAuthenticate()` rather than assuming.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20

Mid-Level

BiometricPrompt + CryptoObject · Passkeys · Play Integrity · Sensitive screens



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Bind the fingerprint to a key

```
val spec = KeyGenParameterSpec.Builder(
    "payment_key", KeyProperties.PURPOSE_SIGN,
)
    .setAlgorithmParameterSpec(ECGenParameterSpec("secp256r1"))
    .setDigests(KeyProperties.DIGEST_SHA256)
    .setUserAuthenticationRequired(true)
    // timeout 0 = a fresh strong biometric for every use
    .setUserAuthenticationParameters(
        0, KeyProperties.AUTH_BIOMETRIC_STRONG) // API 30+
    .setInvalidatedByBiometricEnrollment(true)
    .setAttestationChallenge(enrollmentNonce)   // Q9
    .build()

KeyPairGenerator.getInstance(
    KeyProperties.KEY_ALGORITHM_EC, "AndroidKeyStore",
).apply { initialize(spec) }.generateKeyPair()
```

Enrollment: the public key goes to your server once. **Every payment:** the server sends a challenge, the user authenticates, the Keystore signs, the server verifies.



Esmaeeil Moradi
@esmaeeil-moradi700

Q4. Sign with a CryptoObject

```
val signature = Signature.getInstance("SHA256withECDSA")
    .apply { initSign(privateKey) } // unusable until the prompt

val cb = object : BiometricPrompt.AuthenticationCallback() {
    override fun onAuthenticationSucceeded(
        result: BiometricPrompt.AuthenticationResult,
    ) {
        val s = checkNotNull(result.cryptoObject?.signature)
        s.update(serverChallenge)
        viewModel.submitPayment(s.sign())
    }
}
BiometricPrompt(activity, mainExecutor, cb)
    .authenticate(info, BiometricPrompt.CryptoObject(signature))
```

INTERVIEW TRAP

"`setUserAuthenticationRequired(true)` *is enough."* — With a timeout above 0, the key works for that many seconds after **any** unlock — no prompt from you. Per-transaction proof needs timeout 0 **and** a `CryptoObject`. (Below API 30: `setUserAuthenticationValidityDurationSeconds(-1)`.)

FOLLOW-UP

"The user added a new fingerprint — what now?" — `initSign()` throws `KeyPermanentlyInvalidatedException`. Re-verify the user another way, create a new key and re-enroll it.



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Passkeys with Credential Manager

```
suspend fun signIn(activity: Activity, requestJson: String) {
    val request = GetCredentialRequest(
        listOf(GetPublicKeyCredentialOption(requestJson)),
    )
    try {
        val result = CredentialManager.create(activity)
            .getCredential(activity, request)
        val cred = result.credential
        if (cred is PublicKeyCredential) {
            api.finishLogin(cred.authenticationResponseJson)
        }
    } catch (e: GetCredentialCancellationException) {
        // user closed the sheet: stay quiet, no error dialog
    } catch (e: NoCredentialException) {
        showPasswordOrSignUp()
    }
}
```

- `requestJson` is the WebAuthn challenge **from your server**; the server also verifies the response and then issues session tokens (Day 12)
- The same API returns passwords and Sign in with Google (`GetGoogleIdOption`) in one bottom sheet



Esmaeeil Moradi
@esmaeeil-moradi700

Q5. Why passkeys win

	Password	Passkey
Server stores	A hash of a shared secret	**Only a public key**
Phishing	The user can type it into any site	Bound to your domain — a fake site gets nothing
Breach impact	Hashes get cracked and reused	Nothing reusable leaks
UX	Remember and type	Unlock the device: fingerprint, face, PIN

- Link app and domain with **Digital Asset Links** (`assetlinks.json`) — using the **app-signing** certificate fingerprint from Play Console
- Legacy Google Sign-In for Android and Smart Lock for Passwords are deprecated in favor of Credential Manager

INTERVIEW TRAP

"With passkeys, the fingerprint goes to the server." — Never. The biometric only unlocks the private key **on the device**; the server receives a signature over its challenge.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. Play Integrity: getting a token

```
private val integrity =
    IntegrityManagerFactory.createStandard(app)
private var provider: StandardIntegrityTokenProvider? = null

// Once, early — preparing can take a while
suspend fun prepare() {
    provider = integrity.prepareIntegrityToken(
        PrepareIntegrityTokenRequest.builder()
            .setCloudProjectNumber(CLOUD_PROJECT_NUMBER)
            .build()
    ).await()
}

// Per high-value action, bound to that action's payload
suspend fun tokenFor(payload: String): String =
    checkNotNull(provider).request(
        StandardIntegrityTokenRequest.builder()
            .setRequestHash(sha256Hex(payload))
            .build()
    ).await().token()
```

The app never reads the verdict — it forwards the token with the request. If the provider expires, prepare it again.

INTERVIEW TRAP

"We use the SafetyNet Attestation API." — Retired. Play Integrity replaced it, with app, device and account signals in one token.



Esmaeeil Moradi
@esmaeeil-moradi700

Q6. What the verdict tells your server

Field	Meaning
`appIntegrity.appRecognitionVerdict`	`PLAY_RECOGNIZED`: the exact binary you uploaded to Play
`deviceIntegrity.deviceRecognitionVerdict`	Labels such as `MEETS_BASIC_INTEGRITY`, `MEETS_DEVICE_INTEGRITY`, `MEETS_STRONG_INTEGRITY`
`accountDetails.appLicensingVerdict`	`LICENSED`: the user got the app from Google Play
`environmentDetails` (optional)	Risky apps running; Play Protect status

INTERVIEW TRAP

"Decode the verdict in the app and block." — The token is encrypted and signed; your **server** decodes it through Google Play and decides. A client-side "if" is one patch away from `true`.

FOLLOW-UP

"And users without Google Play?" — No Play Store or Play services, no verdict: alternative stores, some regions. Plan a fallback — key attestation (Q9) and step-up checks — instead of a hard block.



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Protecting sensitive screens

```
class RecoveryPhraseActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // No screenshots, recordings or recents thumbnail
        window.addFlags(WindowManager.LayoutParams.FLAG_SECURE)
        if (Build.VERSION.SDK_INT >= 31) {
            // requires HIDE_OVERLAY_WINDOWS (normal permission)
            window.setHideOverlayWindows(true)
        }
        setContent { RecoveryPhraseScreen() }
    }
}

// A secret dialog on an otherwise normal screen
Dialog(
    onDismissRequest = onDismiss,
    properties = DialogProperties(
        securePolicy = SecureFlagPolicy.SecureOn),
) { OtpContent() }
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q7. Attacks and platform defenses

Attack	Platform defense
Screenshot, recording, casting	`FLAG_SECURE`; callbacks for screenshots (API 34) and recording (API 35)
Tapjacking — an overlay tricks the user into tapping	Android 12+ blocks most untrusted pass-through touches; `filterTouchesWhenObscured`; hide overlays (API 31)
Accessibility malware reading the screen	`accessibilityDataSensitive` (API 34); restricted settings for sideloaded apps (Android 13)
Keyboard learning a PIN	Password `inputType`; `IME_FLAG_NO_PERSONALIZED_LEARNING`

INTERVIEW TRAP

"Build a custom secure keyboard." — Last year's advice. An in-app keyboard is more code to get wrong; the platform flags above already stop learning and capture.

INTERVIEW TRAP

"*`FLAG_SECURE` on the Activity covers everything."* — A View-based `Dialog` or `PopupWindow` is a **separate window** and doesn't inherit it. Compose dialogs inherit by default (`SecureFlagPolicy.Inherit`).



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20



Senior

Verifying integrity on the server · Key attestation · Responding to tampering



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Verifying the integrity verdict on the server

```
{
  "requestDetails": {
    "requestPackageName": "com.example.bank",
    "requestHash": "9f2c…",
    "timestampMillis": "1790000000000"
  },
  "appIntegrity": {
    "appRecognitionVerdict": "PLAY_RECOGNIZED",
    "certificateSha256Digest": ["6a6a…"]
  },
  "deviceIntegrity": {
    "deviceRecognitionVerdict": ["MEETS_DEVICE_INTEGRITY"]
  },
  "accountDetails": { "appLicensingVerdict": "LICENSED" }
}
```

1. Decode the token with Google Play's `decodeIntegrityToken` API — on the **server**
2. `requestPackageName` is yours; `timestampMillis` is fresh
3. `requestHash` equals the hash of **the request you just received** — a token for "view balance" can't authorize "send money"
4. Only then read the verdicts



Esmaeeil Moradi
@esmaeeil-moradi700

Q8. Turning verdicts into policy

Signals	Decision
`PLAY_RECOGNIZED` + `MEETS_DEVICE_INTEGRITY`	Allow
Only `MEETS_BASIC_INTEGRITY` (e.g. unlocked bootloader)	Allow low-risk actions; step up for transfers
`UNRECOGNIZED_VERSION`	Ask to update from the store; block high-risk actions
Empty device verdict / no Google Play	Fall back to key attestation + OTP — don't hard-fail

INTERVIEW TRAP

"Check integrity once at login." — A session lasts days; the device can change in minutes. Request a token **at the high-value action**, bound to it with `requestHash`.

KEEP THE POLICY ON THE SERVER

The table above lives in backend config, not in the APK. Tighten it during an attack without shipping a release — and remember Play Integrity has a daily request quota, so spend calls on actions that matter.



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. Key attestation: proof from the secure hardware

```
// Key created with setAttestationChallenge(nonce) — see Q4
val ks = KeyStore.getInstance("AndroidKeyStore")
    .apply { load(null) }
val chain = ks.getCertificateChain("payment_key").map {
    Base64.encodeToString(it.encoded, Base64.NO_WRAP)
}
api.registerKey(chain) // leaf certificate first
```

The server verifies:

- The chain up to a trusted attestation root, and no certificate on Google's **revocation list**
- The **challenge** equals the nonce it issued
- `attestationSecurityLevel` is TEE or StrongBox — not software
- `attestationApplicationId`: your package and signing certificate
- `rootOfTrust`: verified boot state and a locked bootloader
- Key properties: purpose SIGN, user authentication required



Esmaeeil Moradi
@esmaeeil-moradi700

Q9. What attestation does — and doesn't — prove

Attestation comes from **KeyMint/Keystore itself**, not from Play services — which makes it the natural fallback where Play Integrity isn't available.

INTERVIEW TRAP

"Attestation proves the device isn't rooted." — It proves the **key** lives in secure hardware with the properties you asked for, and reports the boot state. It says nothing about what else runs on the device, and leaked attestation keys exist — hence the revocation list. A strong signal, not proof.

INTERVIEW TRAP

"The attestation certificate proves our code wasn't modified." — Last year's version implied this. It identifies the package and signing certificate — a repackaged APK signed with another key shows up; a hooked process on the original APK does not.

FOLLOW-UP

"Why not skip attestation and just trust the enrolled public key?" — Without attestation you can't tell a hardware-bound key from one generated in software by a patched app — or by a script.



Esmaeeil Moradi
@esmaeeil-moradi700

Q10. You detected tampering — now what?

Response	Why
Report the signal, keep the app usable	The server correlates it with integrity verdicts; the attacker doesn't learn which check fired
Step up risky actions: re-auth, OTP, lower limits	Real users on unusual devices still get through
Delay and vary the reaction	An instant crash points straight at the check
Hard blocks only as a **server** decision	One place to tune policy; no release needed
Kill switch and minimum version via remote config	React to an active attack in minutes

INTERVIEW TRAP

"Close the app immediately when the hash doesn't match." — Last year's advice. It punishes false positives and teaches the attacker exactly where to patch.

COMMERCIAL RASP

Tools like DexGuard-class obfuscation, string encryption and hook detection raise the cost — worth it for banking or games. They're still client-side: buy them **in addition to** server checks, never instead.



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20

Architect

Defense in depth · OWASP MASVS & MASTG · Supply-chain security



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. Defense in depth for a payment flow

```
app ── payment + integrity token + signed challenge ──▶
backend
  ├─ decodeIntegrityToken ──▶ Google Play
  ├─ verify signature with the attested public key
  ├─ risk score = integrity + attestation + behavior
  └─ low: allow | medium: step up (OTP) | high: deny
```

- **Hardening:** R8, no secrets in the APK, Part 1's closed doors
- **Hardware:** auth-bound Keystore keys, passkeys
- **Attestation:** Play Integrity, key attestation
- **Server:** risk engine, limits, anomaly detection, audit log



Esmaeeil Moradi
@esmaeeil-moradi700

Q11. If one layer falls

Layer bypassed	What still protects the user
R8 obfuscation read through	No secrets in the APK; server-side rules
Root check hooked	Integrity verdict decoded on the server
Biometric callback faked	No signature without the auth-bound key
Stolen session token	Payments still need a fresh signed challenge
Integrity unavailable	Key attestation + step-up + transaction limits

INTERVIEW TRAP

"We have pinning and root detection, so we're secure." — Each layer alone is bypassable. The design goal is that **no single bypass moves money**.

FOLLOW-UP

"What do you monitor?" — Integrity failures by app version and device model, step-up rates, signature failures. A spike after a release is a bug; a spike from one device farm is an attack.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. OWASP MASVS: the shared checklist

MASVS group	Covers	In this series
STORAGE	Sensitive data at rest, logs, backups	Day 19
CRYPTO	Keystore, algorithms, key management	Day 19
AUTH	Passkeys, biometrics, sessions	Day 20 · Day 12
NETWORK	TLS, pinning	Day 12
PLATFORM	IPC, intents, WebView, UI leaks	Day 19 · Day 20
CODE	Dependencies, input validation, updates	Day 20
RESILIENCE	Obfuscation, integrity, anti-tampering	Day 20
PRIVACY	Data minimization, transparency, user control	Day 19

MASVS says *what* to achieve. **MASTG** (the testing guide) says *how* to test it, with techniques and tools. MASWE catalogs the concrete weaknesses in between.



Esmaeeil Moradi
@esmaeeil-moradi700

Q12. Using it in practice

- Pick a **testing profile**: L1 baseline for every app, L2 for sensitive data (health, finance), R for resistance to reverse engineering, P for privacy
- Map the controls to your backlog and **Definition of Done** — not to a PDF read once a year
- Test with the MASTG: static and dynamic analysis, jadx, Frida, MobSF
- Need independent proof? App Defense Alliance **MASA** assesses apps against MASVS; Google Play can show the result in the Data safety section

INTERVIEW TRAP

"MASVS has levels L1 and L2." — Outdated. MASVS v2 is just the list of controls; the levels moved to the MAS **testing profiles**.

INTERVIEW TRAP

"Resilience makes up for weak storage." — R is **on top of** L1/L2, never instead. Obfuscating code that writes tokens to logcat only hides the bug from you.



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. Supply chain: your build is attack surface

```
// settings.gradle.kts
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        // Internal groups resolve ONLY from the internal repo
        exclusiveContent {
            forRepository {
                maven("https://maven.internal.example.com")
            }
            filter { includeGroupByRegex("com\\.example\\..*") }
        }
    }
}
```

```
# Record checksums; the build fails when an artifact changes
./gradlew --write-verification-metadata sha256 help
```



Esmaeeil Moradi
@esmaeeil-moradi700

Q13. The supply-chain toolbox

Risk	Control
Dependency confusion	`exclusiveContent` repository filtering
Tampered artifact	Gradle dependency verification (`verification-metadata.xml`)
Known-vulnerable versions	Dependabot / Renovate, OSV-Scanner, Play SDK Index warnings
Malicious Gradle wrapper	`distributionSha256Sum` + wrapper validation in CI
Leaked signing key	Play App Signing: Google holds the app-signing key; a lost upload key can be reset
Unknown inventory	An SBOM (e.g. CycloneDX) per release

INTERVIEW TRAP

"Dependencies are someone else's code, so someone else's problem." — They run with your UID and your permissions (Day 19). Gradle plugins are code too — and they run on CI, next to your signing secrets.

SENIOR ANSWER IN ONE LINE

"Client checks raise the cost, hardware keys make the proof, the server makes the decision — and the build that produces all of it is part of the attack surface."



Esmaeeil Moradi
@esmaeeil-moradi700

DAY 20 · ARCHITECT

Day 20 — cheat sheet

- R8 shrinks and renames; it doesn't encrypt strings. No secrets in the APK.
- Root and tamper checks are bypassable signals: report them, don't crash on them.
- With Play App Signing, compare against the **app-signing** certificate, not your upload key.
- Biometrics without a `CryptoObject` is just a callback; bind it to an auth-bound Keystore key (timeout 0).
- Passkeys via Credential Manager: public key on the server, phishing-resistant, Digital Asset Links required.
- Play Integrity replaced SafetyNet: bind the token with `requestHash`, decode and decide **on the server**.
- No Google Play? Fall back to key attestation + step-up — and check the revocation list.
- Sensitive screens: `FLAG_SECURE`, overlay protection, `accessibilityDataSensitive`; no custom keyboards.
- MASVS = what, MASTG = how; L1/L2/R/P are testing profiles now.
- Supply chain: repository filtering, dependency verification, wrapper checksum, Play App Signing, SBOM.

Day 21 is next → Build System & Gradle, part 1. Save this post and follow along.



Esmaeeil Moradi
@esmaeeil-moradi700